Architecture Documentation

DEVision - Job Applicant Subsystem

Samson Squad - Job Applicant Team

November 28th 2025

Team Members:

Truong Phuoc Huy - s3891442

Hong Thieu Kiet - s3993986

Ngo Quang Hieu - s3820591

Tran Xuan Hoang Dat - s3651550

Tran Quang - s3976073

Introduction

The Job Applicant subsystem of DEVision is designed with a modern, scalable, and maintainable software architecture that ensures high performance, flexibility, and long-term extensibility. To meet the system's functional and non-functional requirements, the subsystem adopts a combination of **Modular Frontend Architecture** and a **Microservices Backend Architecture**, enhanced by an event-driven communication model and container-based deployment.

On the **Frontend**, the subsystem is implemented using a **Modular Architecture**, where each feature—such as Authentication, Profile Management, Job Search, Application Management, and Subscription Management—is isolated as an independent module. This architectural approach improves maintainability and promotes separation of concerns by decoupling UI, hooks, and services into cohesive units. It also provides extensibility, allowing future modules to be added with minimal impact on the existing codebase.

On the **Backend**, the subsystem follows a full **Microservices Architecture** built with **Node.js** and **TypeScript**, applying **SOLID principles** across all service components to ensure clean abstractions, reduced coupling, and easy testability. Each service operates independently and manages its own MongoDB collection, ensuring autonomy and high scalability. The microservices communicate using **Apache Kafka**, enabling reliable asynchronous event streaming for features such as job application events, applicant profile updates, and premium notification triggers.

A centralized **API Gateway** serves as the single entry point for all frontend requests, handling routing, authentication, rate limiting, and cross-service orchestration. Along with this, **Service Discovery** is implemented to dynamically register, locate, and load-balance microservices, eliminating the need for hardcoded service endpoints and supporting horizontal scaling.

To ensure portability and consistent execution across environments, the entire subsystem is containerized using **Docker**, allowing each microservice, Kafka broker, API Gateway, and frontend module to be deployed as independent yet interconnected containers.

Finally, the Job Applicant subsystem integrates seamlessly with the **Job Management subsystem**, enabling real-time synchronization of job posts and application data across the platform. This integration enhances system interoperability while maintaining clear subsystem boundaries.

With this combination of technologies—Modular Frontend, Microservices Backend, Kafka, MongoDB, TypeScript, API Gateway, Service Discovery, Docker, and SOLID principles—the Job Applicant subsystem provides a robust, scalable, and future-ready architecture aligned with industry best practices and the technical expectations of the DEVision project.

Squad Data Model

Data Model

Figure 1: Whole system Entity-Relationship Diagram (ERD)

Overview

Our system is split into two sides:

- Job Manager (JM) stores companies and job posts
- Job Applicant (JA) stores applicants, their profiles, media, applications, payments and notifications.

In our design, we use user_id as the main bridge between JA and JM subsystems. We store this in our application and notification data so we can always track the applicant back to the correct job post.

Authentication

- We split authentication data from profile data to make it more secure.
- Password is sensitive data so we do not want it to accidentally appear in normal profile queries.

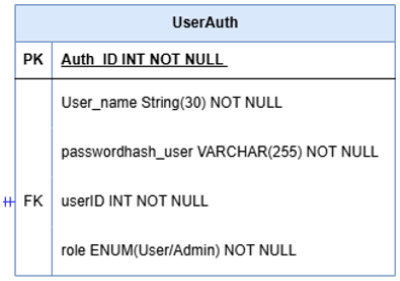

Figure 2: UserAuth table

Profile and portfolio

- Our profile data cover: basic account info, contact details, and education information(degree, institution, GPA)

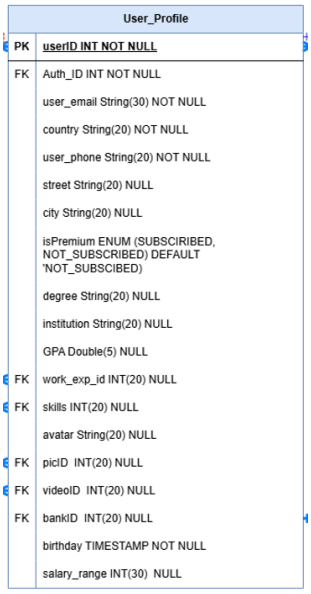

Figure 3: User Profile table

- Work Experience is stored as repeatable entries so users can keep listing their past job(title, duration, description).

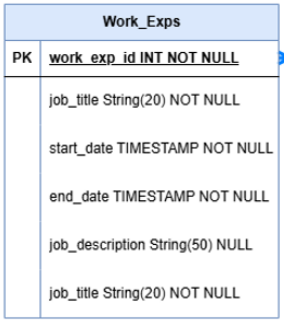

Figure 4: Work experience table

- Technical skills are modelled as tags instead of plain text. This lets us reuse the same tags later in job search.

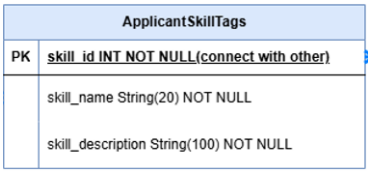

Figure 5: Skill Tags table

- For portfolio activities, we store images and videos in the database and use their media links instead. This will prevent putting raw data media and reduce size.

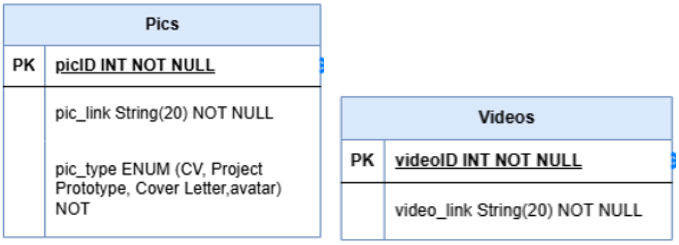

Figure 6, 7: Media table

Job Search and Job Application

- We do not recreate the Job entity. Instead, job data lives in JM in the JobPost table, which contains fields like jod_id, company, title, description, etc. When a user searches, our backend will call the JM's JobPost APIs and uses these fields and the skill tags to implement the search feature in the specification

- When an applicant applies, we create a job application record that tie together the applicant's profile, job_id, the CV/cover letter, the date and the current application status.

JobApplication	
PK	List_id INT NOT NULL
FK	job_id INT NOT NULL
FK	picID INT NOT NULL
	applied_Date DATE DEFAULT CURRENT_DATE
	status ENUM('APPLIED', 'SCREENING', 'INTERVIEW', 'OFFERED', 'REJECTED', 'HIRED') NOT NULL
FK	userID INT NOT NULL

Figure 8: Job Application table

Premium subscription and payment

- In the profile, we have a premium flag that indicates whether a user is currently a premium subscriber.

isPremium ENUM (SUBSCIRIBED, NOT_SUBSCRIBED) DEFAULT 'NOT_SUBSCIBED)

Figure 9: Premium tag field

- We also keep payment records with the user's identity and transaction history.

Squad Container Diagram

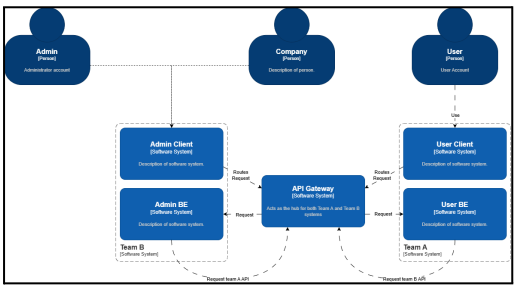

Figure 10: Squad System Container Diagram

Job Applicant Subsystem – Overview

- The subsystem is designed for individuals seeking jobs in the Computer Science domain.
- It is responsible for all applicant-facing functionalities, including:
  - Account registration
  - Login & authentication
  - Profile creation and management
  - Job search & filtering
  - Job application submission

    - Application status tracking
    - Premium subscription and real-time notifications

- Built using a **Modular Frontend** and **Microservices Backend** to ensure maintainability, scalability, and clean separation of concerns.
- Utilizes **Kafka** for event-driven premium notifications (e.g., new job alerts).
- Stores applicant-related data using **MongoDB**.

🔹 Connection Through API Gateway(NodeJS, Typescript)

- The Job Applicant Subsystem does **not** communicate with other subsystems directly.
- All communication goes through the **API Gateway**, including:
    - Fetching public job listings
    - Submitting job applications
    - Searching jobs
    - Synchronizing application status

- The API Gateway handles:
    - Request routing
    - Authentication & token validation
    - Cross-system communication
    - Hiding internal endpoints of other services

🔹 Backend Architecture Highlights(NodeJS, Typescript, Mongoose,Prisma,Kafka,Docker)

- Backend is structured as multiple independent microservices:
    - Application Service
    - Auth Service
    - Profile Service
    - Search Proxy Service
    - Job Posts Service
    - Application Service
    - Matching Profile Service
    - Subscription Service
    - Notification Consumer (Kafka)

- Implements **SOLID principles**, enabling loosely coupled, testable, and maintainable code.
- Each service runs inside a dedicated Docker container for easy deployment and scaling.
- Uses **Service Discovery** so microservices can register and communicate dynamically without hardcoded service URLs.

🔹 Frontend Architecture Highlights(React, Bootstrap, Tailwind ,Redux)

- Developed using a **Modular Frontend Architecture**:
    - Each major feature exists as an independent module
    - Improves maintainability and code clarity

    - Enables adding new modules without affecting existing ones

- Key modules include:
    - Login Module
    - User Profile Module
    - Job List Module
    - Subscription Module
    - Notification Module
    - Job History Module
    - Admin Panel Module

JM Subsystem - Overview (Minimal Mention – Outside Team Scope)

- Used by companies and admins.
- Handles job posting, job updates, and reviewing applicants.
- Only provides **public job data** to the Job Applicant Subsystem through the API Gateway.
- Internal logic and operations are not part of the Job Applicant team's responsibilities.

Frontend

Figure 11: Job Manager Frontend Container Diagram

The **Job Manager Frontend** is divided into container-level modules that manage high-level concerns (security, routing, global state), and feature-level modules that deliver the functionality used by company users. Containers such as Browser Router, Security, State Management, and Layout form the backbone of the system, while feature containers such as Job Post Feature, Applicant Search, and Subscription & Payment implement the core business functions. All communications with the backend occur through a dedicated HTTP Unit, which encapsulates API URLs, headers, token injection, and error handling. This avoids duplication and ensures consistent communication across all modules.

Backend

Figure 12: Job Manager Backend Container Diagram

The **Job Manager Backend** adopts a Modular Monolith architecture using Spring Boot, where each major business capability is implemented as an independent backend module within a shared application. Incoming requests from the frontend first pass through the Filter and Security containers, where request preprocessing, token validation, and authorization checks occur. Once validated, requests are routed to the appropriate module: the Auth module handles authentication and registration, the Company module manages company profiles and employer operations, the JobPost module processes job posting workflows, and the Subscription and Payment modules handle subscription features and premium plan payments. These modules interact with a centralized MongoDB database, which stores authentication data, job posts, company information, subscriptions, and payment records.

The backend also integrates with external services to support advanced functionality. The Google Auth Service is used for third-party authentication flows, while Stripe processes premium subscription payments. Additionally, the Job Manager backend communicates with the DEVision Job Application subsystem to retrieve applicant profiles and update their application statuses. This modular backend structure ensures clear separation of responsibilities, secure request handling, and efficient interaction between internal modules and external systems while maintaining consistency within a single codebase and database.

System Architecture (C4 Model)

BACKEND

Backend Container Diagram

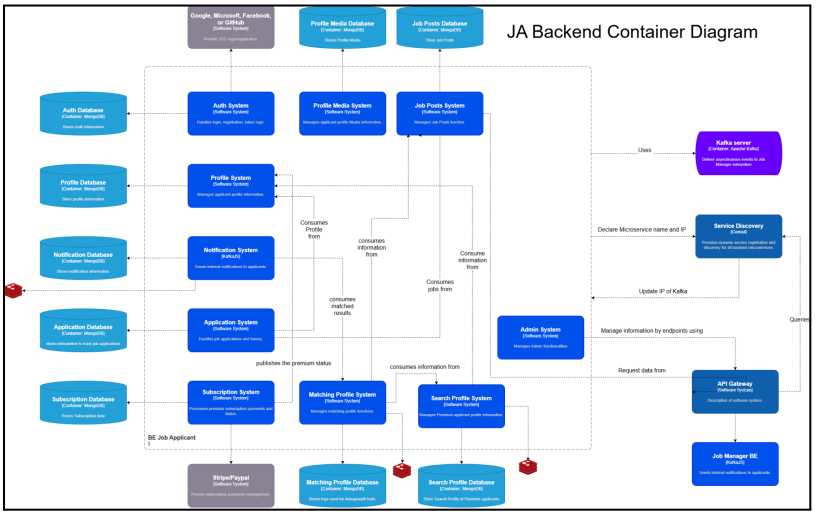

Figure 13: [Backend Microservice Container Diagram]

1. **API Gateway (System)**
    - Clients will use this to connect to the backend using IPs and names provided by service discovery.
2. **Kafka server:**
    - Microservices can publish to a topic on Kafka, other microservices can subscribe to a topic to receive the determined data.
    - Allows the **Real-Time Matching Module** to immediately receive job-related events and trigger matching logic to send notifications to users.
3. **Microservices**
    - Each system business logic is implemented as an independent service. They communicate through Kafka, update their own IPs and names through Service Discovery. Each of them is deployed and can be scaled independently.
    - List of the Microservices:
        - **Auth System**: manages login/registration, JWT tokens and password hashing.
        - **Profile System**: provides applicant profiles management and profiles details for other systems.

        - **Notification System**: provides email/in-app/real-time notifications.
        - **Application System**: provides job applications, CVs, application status, feedback.
        - **Subscription System**: provides premium activation management, premium status of applicants to other services.
        - **Profile Media System**: provides images and videos management
        - **Job Posts System**: stores job postings.
        - **Matching Profile System**: provides job match status to **Notification System** for premium users.
        - **Search Profile System**: provides the premium profiles of premium users.
4. **Databases**
    - Each system has its own MongoDB database to store data, which is only exposed by Kafka only.
    - Databases:
        - **Auth Database**: stores User credentials (hashed passwords), OAuth tokens, role and permission mappings
        - **Profile Database**: stores applicant profiles.
        - **Notification Database:** stores notification records, status and templates.
        - **Application Database:** stores application contents (IDs, status, responses).
        - **Subscription Database:** stores subscription related contents(plan, payment status, history, start/end time).
        - **Profile Media Database:** stores file metadata(URL, size, type, user ID).
        - **Job Posts Database:** stores job title, description, requirements, salary, location, experience level.
        - **Matching Profile Database:** stores logs and contents necessary for debugs/audit trails.
        - **Search Profile Database:** stores premium profiles.
5. **Redis Cache**
    - A caching layer used for services requiring speed, caching, or real-time operations.
    - Redis is used in the systems:
        - **Notification system**: stores unread notifications and history, provides rate limiting to prevent spam notifications.
        - **Matching Profile System**: provides fast scoring & filtering for the matching profiles with the jobs.
        - **Search Profile System**: improves search latency for searching features.

Microservice Diagram:

1. Auth Microservice:

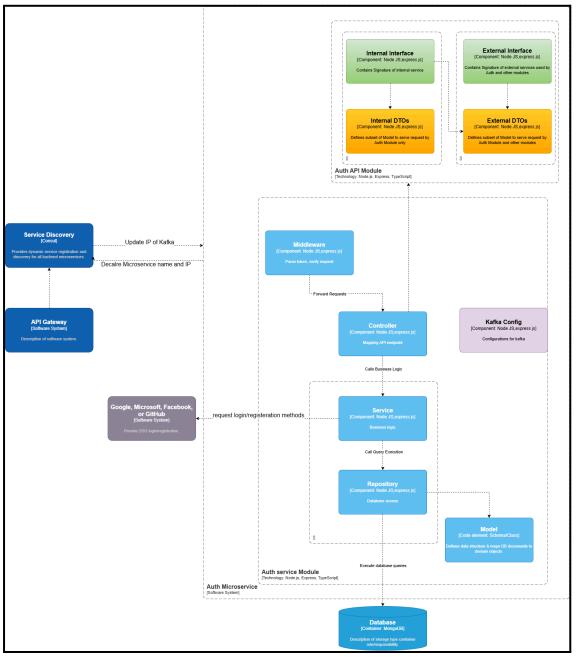

Figure 14: Auth Component Diagram

The Auth service is responsible for all authentication and authorization operations within the system. It manages user registration, login, password hashing, JWT token generation, token validation, and user session security. It also handles role-based access control and provides authentication data to other microservices.

- **API Gateway:** We use Api Gateway to get requests from clients: login, register, refresh access tokens, validate authentication.
- **Controller:** Receive requests from API Gateway, call the Service to do the function and return the response back to the client
- **Service:** Hold the functions to do the work, access the repository and database (hashes and verifies passwords, generates JWT access tokens and refresh tokens, validates tokens and handles token expiration, calls the Repository to read/write user auth data, Manages user roles and permissions, integrates with third-party identity providers: Google OAuth, Facebook OAuth)
- **Repository:** query the database and save/update/delete the entities. Also define the **Model** of the data that save in the database
- **Database:** Save the data of the microservice. Save user hashed credentials , refresh tokens, user roles and access rights
- **Kafka:** is used to send authentication-related events to other microservices if required.

2. Subscription Microservice:

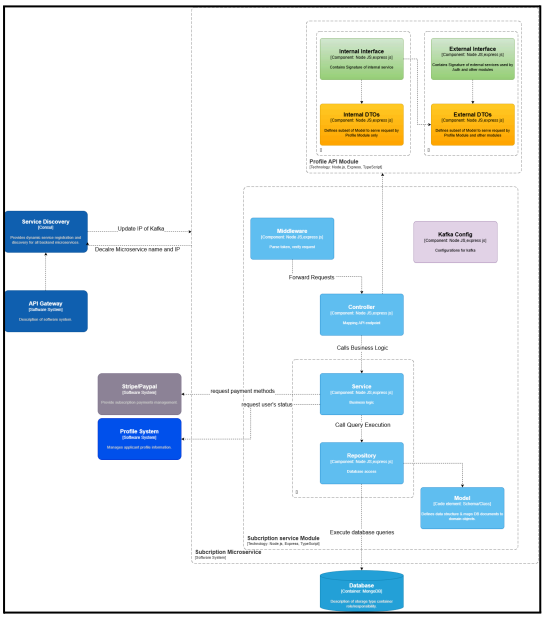

Figure 15: Subscription Component Diagram

The Subscription Microservice is the microservice that deals with application subscription. It handle the money transaction for when the user subscribe to the Premium service, send the Premium status of the user to Profile Microservice and save the transaction data to the database

- **API Gateway:** We use Api Gateway to get request from client: Request to subscribe for Premium, request to pay for Premium
- **Controller:** Receive requests from API Gateway, call the Service to do the function and return the response back to the client
- **Service:** Hold the functions to do the work, call Payment Provider API, access the repository and database.
- **Repository:** query the database and save/update/delete the entities. Also define the **Model** of the data that save in the database
- **Database:** Save the data of the microservice. Save transaction data
- **Kafka:** Communicate between microservices through events. Send Premium status of the user to Profile Microservice

3. Profile Microservice:

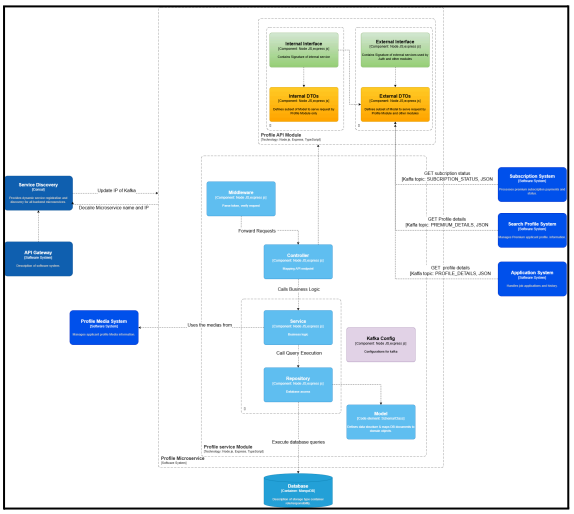

Figure 16: Profile Component Diagram

The Profile Microservice is the microservice that deals with application profiles. It handles all the user profiles and portfolios. It gets videos and pictures from the Profile Media Microservice and send user profile to Application Microservice

- **API Gateway:** We use Api Gateway to get requests from clients: create new users, update user profile and portfolio, …
- **Controller:** Receive requests from API Gateway, call the Service to do the function and return the response back to the client
- **Service:** Hold the functions to do the work, access the repository and database.
- **Repository:** query the database and save/update/delete the entities. Also define the **Model** of the data that save in the database
- **Database:** Save the data of the microservice. Save profile data
- **Kafka:** Communicate between microservices through events

4. Notification Microservice:

Figure 17: Notification Component Diagram

The Notification Microservice is the microservice that deals with application notification. It get the job data from Match Profile Microservice and create notifications for Premium User

- **API Gateway:** We use Api Gateway to get request from client
- **Controller:** Receive requests from API Gateway, call the Service to do the function and return the response back to the client
- **Service:** Hold the functions to do the work, access the repository and database.

- **Repository:** query the database and save/update/delete the entities. Also define the **Model** of the data that save in the database
- **Database:** Save the data of the microservice. Save the Notification data
- **Kafka:** Communicate between microservices through events

5. Application Microservice:

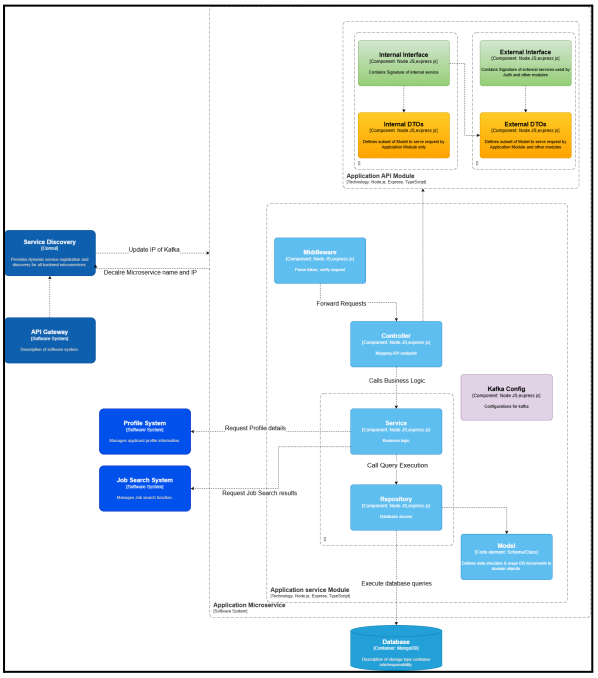

Figure 18: Application Component Diagram

The Application Microservice is the microservice that deals with main application services. It handles job applications, CVs get the job status and feedback.

- **API Gateway:** We use Api Gateway to get request from client
- **Controller:** Receive requests from API Gateway, call the Service to do the function and return the response back to the client
- **Service:** Hold the functions to do the work, access the repository and database.
- **Repository:** query the database and save/update/delete the entities. Also define the **Model** of the data that save in the database
- **Database:** Save the data of the microservice.
- **Kafka:** Communicate between microservices through events

6. Profile Media Microservice:

Figure 19: Profile Media Component Diagram

- Purpose: manage all media attached to the applicant's profile(videos and images)
- Incoming communication:
    - HTTP request from the frontend via API gateway to create/upload/update/delete any images/videos.
- Outgoing communication:
    - Read/write media metadata into the database as URL.
    - Provide media URL to Profile service when a full profile is rendered

7. Search Profile Microservice:

Figure 20: Search profile Component Diagram

- Purpose: Manages saved search profiles for premium applicants: skill tags, locations, salary range, job titles, etc., which are used for matching jobs.
- Incoming communication:
    - HTTP requests from the frontend via API Gateway to create/update/fetch a user's search profile.
    - Internal calls from the Profile System (through Gateway) to read basic profile details when a search profile is created or validated.
- Outgoing communication:
    - Stores search profile data in its own Search Profile database.
    - Uses Redis cache to speed up lookups of search profiles when matching jobs.
    - Publishes saved search profiles for premium users to Kafka topic SEARCH_PROFILE, which is consumed by the Matching Profile microservice during its matching process.

8. Matching Profile Microservice:

Figure 21: Matching Profile Component Diagram

- Purpose: Runs the job applicant matching logic for premium users. It takes their skill tags and saved search profiles, compares them with available job posts, and produces matched job lists.
- Incoming communication
    - HTTP requests from the frontend via API Gateway.
    - Internal calls to the Search Profile System to retrieve each user's stored search profile.
    - Consumes Job data from kafka published by Job Post microservice.
- Outgoing communication
    - Persists matching results in its own Matching Profile database and uses Redis as a cache for frequently accessed matches.

- Sends matched profile data to the Notification System via Kafka.
- Returns matched job lists to the frontend through the API Gateway.

9. Job Post Microservice:

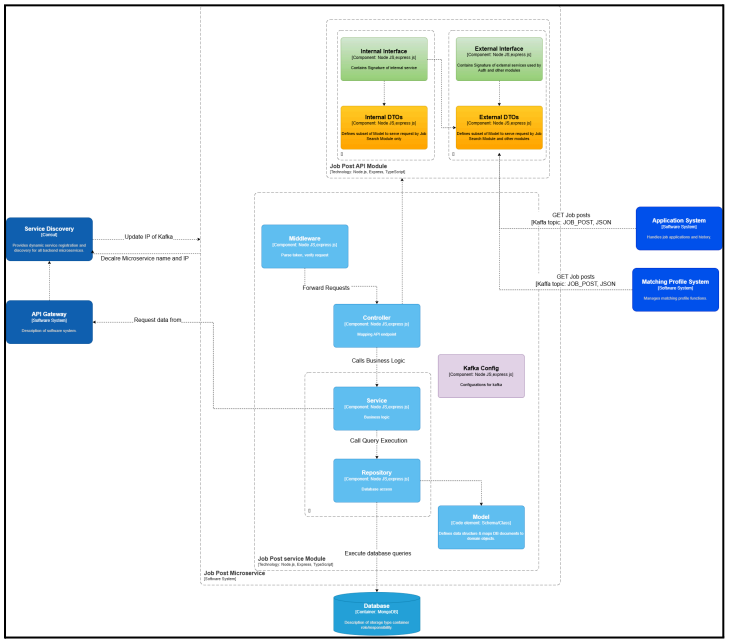

Figure 22: Job Post Component Diagram

- Purpose: Keeps the list of job posts that applicants can browse from the JA side.
- Incoming communication
  - From JA frontend:
    - Requests to list/filter job posts
    - Requests to view details of a specific job
- Outgoing communication
  - To JA frontend:
    - Return job list/details based on request through API gateway
  - To other services via Kafka
    - Whenever a job is created, Job Post publish the job data to the Kafka topic
    - Services that need job data subscribe to the topic.
  - Reads/writes job post records in its own database
  - Publishes match results to Kafka topic JOB_POST. The Notification System consumes this topic and turns those matches into job match notifications for premium users

10. Admin Microservice:

Figure 23: Admin Component Diagram

This service is responsible for managing administrative functions in the system. It handles role-based access control for administrative users and communicates with other microservices to perform administrative tasks.

- **API Gateway:** We use API Gateway to get requests from clients: view and deactivate accounts, view and delete job posts.
- **Controller:** Receive requests from API Gateway, validates admin credentials and permissions call the Service to do the function and return the response back to the client
- **Service:** Hold the functions to perform validation, call downstream microservices, log actions.
- **Repository:** query the database and handle saving admin accounts, audit logs.
- **Database:** Save the data of the microservice. Save admin accounts and logs.

Frontend

Overview

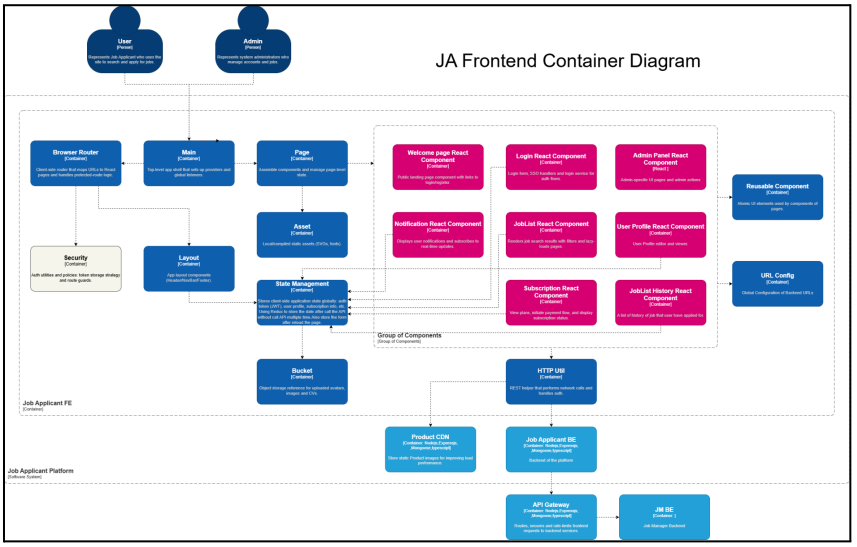

Figure 24: Frontend Container Diagram

The JA Frontend is designed using a **modular and scalable architecture** that emphasizes clean separation of concerns, maintainability, and reusability. Built with **React** for UI rendering and **Redux** for global state management, the system provides a structured way for users and administrators to interact with the platform effectively. The frontend acts as the presentation layer of the system, responsible for rendering interfaces, handling user input, managing client-side state, and communicating securely with the backend through the API Gateway.

At the top level, the application is structured around the **Browser Router**, which defines the navigation across major pages such as Welcome, Login, Job List, Profile, Subscription, and Admin interfaces. The **Main** and **Layout** modules manage the global UI structure, including common components like headers, navigation bars, and footers. A dedicated **Security** layer

(route guard) ensures that protected pages can only be accessed by authenticated users, while also distinguishing access levels based on user roles.

Each major feature in the JA system is implemented as a **self-contained module**, keeping the architecture modular and organized. For example, the Authentication module manages login and registration logic, the Job List module handles job browsing and filtering, and the Profile module manages user information and media display. These modules sit inside the “Group of Components” boundary shown in the diagram. This approach ensures that individual features can be developed, updated, or tested independently without impacting other parts of the system.

To promote UI consistency and reuse, the application uses a **Reusable Component Library**. This library contains essential UI building blocks such as buttons, input fields, modal dialogs, loaders, and card components. Following atomic design principles, these components are kept highly reusable, framework-agnostic, and easily testable. This common component base improves development speed, reduces code duplication, and maintains a uniform visual style throughout the application.

Global state management is handled using **Redux**, chosen for its predictable data flow and suitability for complex applications. Redux stores key data such as authentication tokens, user profiles, job listings, subscription status, and application-wide UI flags. State transitions follow a strict unidirectional flow, making application behavior easier to debug and reason about. Redux slices are organized by domain, ensuring each feature manages its own data responsibly while still contributing to the shared global state. For data fetching, the frontend communicates with the backend through the **HTTP Util** module, which standardizes API calls, injects authorization headers, manages error handling, and routes all requests to the API Gateway.

An important aspect of the JA Frontend is its integration with the **Product CDN**, which serves as the storage for user avatars, images, resumes, and platform media such as videos. Instead of storing files directly in the frontend, the backend provides secure URLs or presigned upload links, ensuring efficient media handling while keeping the frontend lightweight. This design improves performance, reduces bandwidth usage on API services, and ensures fast delivery of static assets to end users.

Frontend Component Diagram explanation

Login React Component

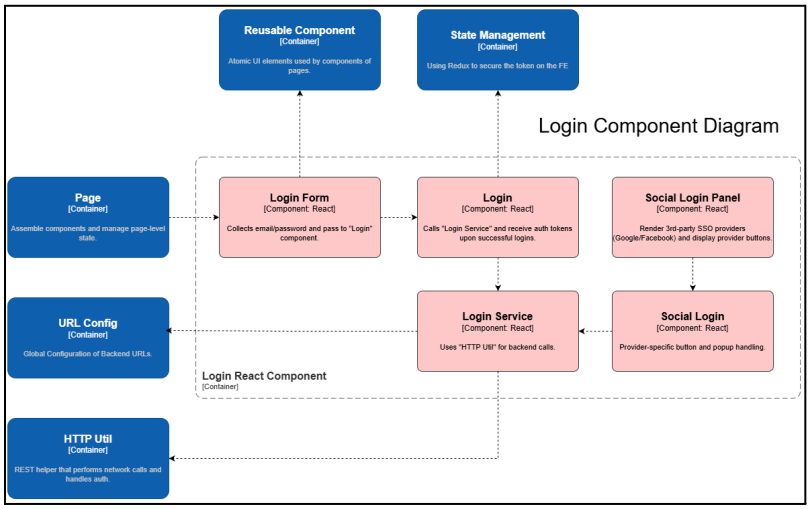

Figure 25: Frontend Login Component Diagram

The **Login React Component** implements authentication UI and logic for the whole Job Applicant Frontend. It supports email/password login, third-party Single Sign-on (SSO) authentication (Google/Microsoft), token storage (JSON Web Token - JWT), page refresh mechanics and integration with global state using Redux. The package follows frontend componentization rules: Presentation, Hooks, Service layer, and Separate styles.

Login Form (Component: React)

- *Purpose:* collect user credentials (email & password), perform client-side validation, show field errors.
- *Responsibilities:* Validate inputs, display loading and errors.
- *Interactions:* **Login** component calls and **Reusable Components** for buttons and inputs.

Login (Component: React)

- *Purpose:* manage the authentication flow and UI state around login actions.
- *Responsibilities:* call “Login Service”, dispatch Redux actions on successful/failed login attempts, redirects on successful login, display cooldown UI if applicable.
- *Interactions:* **Login Service** calls, **State Management** (Redux), **Page Container** (allows redirection) and **HTTP Util**.

Login Service (Module / Service Layer)

- *Purpose:* Centralize API calls and auth logics (login, refresh, logout).
- *Responsibilities:* handles login credentials, call backend endpoints through **HTTP Util.**

- *Interactions:* **HTTP Util**, **State Management**, and **Page** for redirections.

Social Login Panel (Component: React)

- *Purpose:* Render **SSO provider** buttons and start **external OAuth popups** as well as redirects.
- *Responsibilities:* Open provider sign-in popup, monitor login attempt results, forward authorization codes to backend if necessary.
- *Interactions:* **Social Login** subcomponent and **Login Service** to complete server-side handshake.

Social Login (Component: React)

- *Purpose:* provider-specific handlers (Google/Microsoft) — popup lifecycles and results handling.
- *Responsibilities:* Open/monitor popups, close on success, return provider tokens/codes to **Login Service.**

State Management (Redux)

- *Purpose:* Global store for authentication and user session, allowing consistent authenticated state access across the platform.
- *Responsibilities:* Holds refresh tokens and access tokens ;provides selectors and logout/refresh actions.
- *Interactions:* **Login** and **Login Service** (dispatches), **HTTP Util** (read tokens), route guards and pages.

All service components rely on **two shared modules**:

- **HTTP Util** standardizes API communication by attaching JWT access tokens to requests, handling REST errors, and managing token refresh operations. If a request fails due to an expired access token, HTTP Util prompts Login Service to perform a refresh token exchange before retrying the request.
- **URL Config** provides consistent backend endpoints references across development, staging, and production environments.

JobList React Component

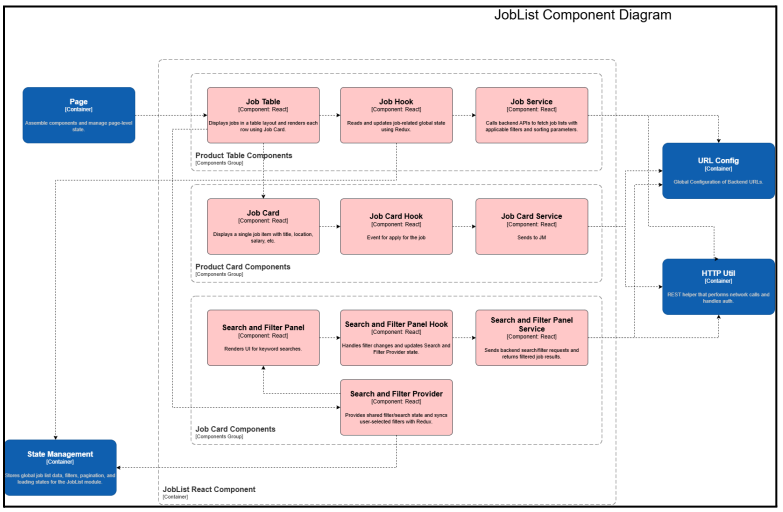

Figure 26: Frontend Job List Component Diagram

The JobList component is one of the core modules of the JA Frontend and is designed following a modular structure that separates presentation, business logic, data fetching, and global state management. This design ensures the module remains scalable, maintainable, and aligned with the frontend's overall architecture based on React, Redux, and service-oriented communication patterns.

At the presentation layer, the JobList module includes the **Job Table**, **Job Card**, and **Search and Filter Panel** components.

- **Job Table** is responsible for rendering job results in a structured table layout. It loops through job data received from the logic layer and displays each job entry using the Job Card.
- **Job Card** displays key job information such as title, location, salary, and company details. This component is strictly visual and does not contain business logic. Its main purpose is to present job data and expose interaction points such as "View Details" or "Apply Now."
- **Search and Filter Panel** provides a user interface for keyword searches and filtering options including position, salary range, job type, or location.

Beneath the UI layer, the module uses a set of **React Hooks** to handle user interaction and flow control:

- The **Job Hook** manages the lifecycle of the JobList module. It retrieves active filters from Redux, triggers the Job Service to fetch job listings from the JM backend system, and updates the global state using Redux actions. It also manages loading states and error handling, making it the central orchestrator of job data.
- The **Job Card Hook** manages interactions arising from the Job Card, specifically "apply to job" events. When the user applies for a position, the hook triggers the Job Card Service to send an application request to the backend. This separation ensures that the Job Card remains a pure UI component while all application logic is isolated within the hook.
- The **Search and Filter Panel Hook** processes user input inside the filter panel, updates the local filter state, and synchronizes user-selected filters with Redux. This ensures that all parts of the JobList module use consistent filtering criteria.

API communication is encapsulated within dedicated **service components**, each handling a specific backend-related responsibility:

- **Job Service** communicates with the JM Backend system to fetch job listings based on filters, search keywords, pagination, and sorting parameters.
- **Job Card Service** handles job application requests. When a user applies for a job, this service sends the application payload (e.g., applicant ID, job ID, timestamp, and any required metadata) through the API Gateway to the backend.
- **Search and Filter Panel Service** processes advanced filtering operations and returns filtered results if server-side filtering is required.

All service components rely on two shared modules:

- **HTTP Util**, which standardizes API calls by attaching authorization headers, handling REST errors, and managing token-based authentication.

- **URL Config**, which provides centralized configuration for backend endpoints, ensuring consistent routing across development, staging, and production environments.

To support consistent state across the module, the JobList component integrates with **State Management (Redux)**. Redux stores global job list data, filters, pagination details, loading states, and error statuses. The Job Hook dispatches Redux actions to update this global job list state, while the Search and Filter Provider syncs filter selections with Redux to keep filtering behavior consistent.

The **Search and Filter Provider** uses React Context to temporarily store local filter states used within the filter panel. It also synchronizes these values with Redux, ensuring that Job Hook always fetches data using the most up-to-date filter criteria. This combination of React Context for UI-level state and Redux for global application state allows the module to maintain responsiveness while avoiding unnecessary re-renders across unrelated components.

JobList History React Component

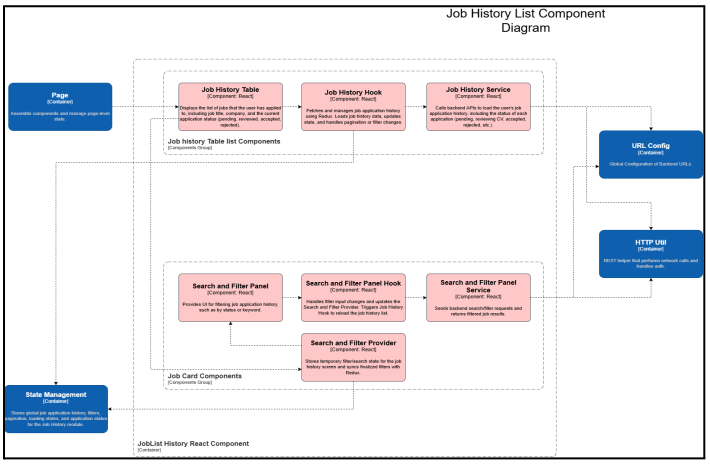

Figure 27: Frontend Job History List Component Diagram

The Job History component is designed using the same modular structure as the other JA Frontend modules, separating presentation, logic, data fetching, and state management. This ensures the feature remains maintainable and aligns with the React–Redux architecture.

At the presentation layer, the module includes the **Job History Table** and **Search and Filter Panel**.

- **Job History Table** displays the list of jobs the user has applied to, including job information and application status (pending, reviewing CV, accepted, rejected).
- **Search and Filter Panel** allows users to filter their application history by keywords or status.

Below the UI layer, the module uses dedicated React Hooks:

- The **Job History Hook** loads the user's application history by reading filters from Redux, calling the Job History Service, and updating global state.
- The **Search and Filter Panel Hook** handles filter input and synchronizes it with the Search and Filter Provider and Redux.

API communication is supported by service components:

- **Job History Service** retrieves application history from the JA Backend.
- **Search and Filter Panel Service** handles additional server-side filtering if required.

Shared utilities including **HTTP Util** and **URL Config** ensure consistent API behavior and endpoint configuration.

State Management is handled by **Redux**, which stores application history data, filters, pagination, loading states, and errors. The **Search and Filter Provider** maintains temporary filter state and syncs finalized values with Redux for consistent behavior.

User Profile React Component

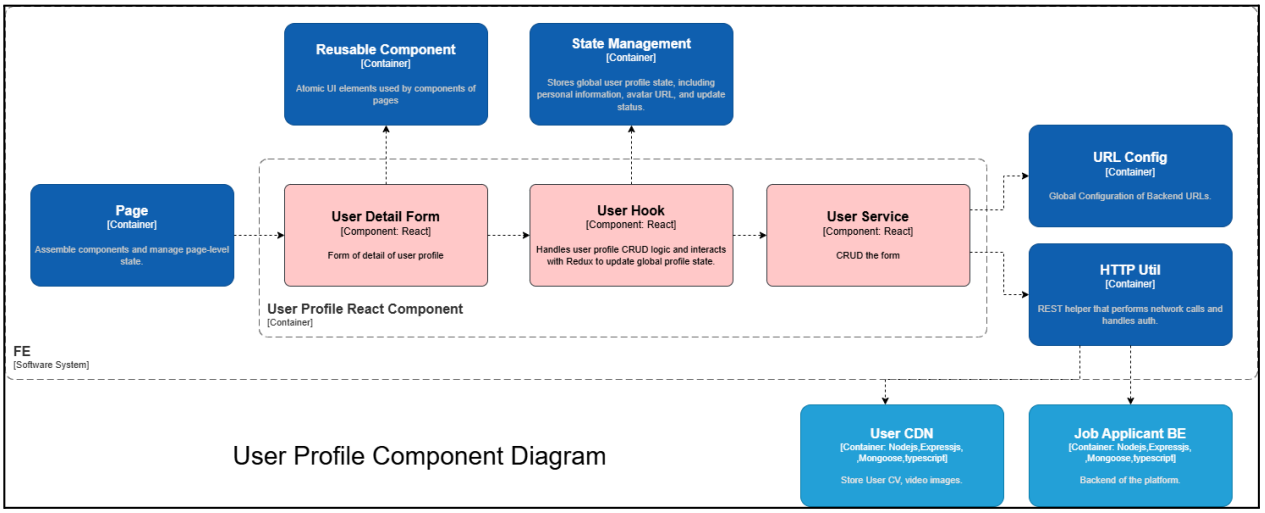

Figure 28: Frontend User Profile Component Diagram

The **User Profile Component** adopts the same modular structure used across the platform, ensuring a clear separation between presentation, logic, service communication, media handling, and global state management. This design supports maintainability, scalability, and predictable data flow within the React–Redux architecture. The module provides an intuitive interface for users to view and update personal information while relying on dedicated hooks and services to handle application logic and backend interactions. Media files such as avatar images or CV documents are processed securely through backend-managed CDN workflows, ensuring consistent and safe access control. Overall, the User Profile module demonstrates a clean layering approach that aligns with modern frontend engineering practices.

Presentation Layer – User Detail Form

The User Detail Form represents the visual layer of the profile page. It displays the user's personal information and captures updates with a UI-focused approach.

Key points:

- Displays user data (name, email, phone number, avatar).
- Provides fields for editing profile details.
- Uses reusable UI components for visual consistency.
- Contains no business logic or API logic—presentation only.

Logic Layer – User Hook

The User Hook encapsulates all profile-related business logic, keeping UI components lightweight and focused.

Key points:

- Manages CRUD operations triggered from the form.
- Validates user input and prepares update payloads.
- Communicates with User Service for profile updates and retrieval.
- Synchronizes profile changes with Redux to ensure consistent global state.
- Ensures the UI always reflects the most updated user data.

Service Layer – User Service

The User Service handles all backend communication using shared utilities to guarantee consistent API behavior.

Key points:

- Sends CRUD requests to the Job Applicant Backend.
- Retrieves updated user profile data.
- Requests pre-signed URLs from backend for avatar or CV uploads.
- Uses HTTP Util for authentication headers and standardized REST behavior.
- Reads API endpoints from URL Config for environment consistency.

Media Handling – User CDN

User media files are stored in the User CDN, but all interactions are strictly mediated through the backend.

Key points:

- Stores user avatar images, CV files, and other media.
- Frontend never interacts directly with the CDN.
- Backend returns pre-signed URLs for secure upload and download.
- Ensures access control, validation, and safe file operations.

State Management – Redux

Redux maintains the global user profile state so that all frontend components use consistent and synchronized information.

Key points:

- Stores personal information, avatar URL, and update status.
- Allows shared components (e.g., header, account menu) to access profile data.
- Updated via dispatches from the User Hook after backend operations.
- Provides predictable unidirectional data flow.

Shared Utilities – HTTP Util & URL Config

These utilities support consistent and secure backend communication throughout the module.

Key points:

- HTTP Util injects authorization tokens and handles errors.
- URL Config centralizes backend endpoint configuration for all environments.

Notification React Component

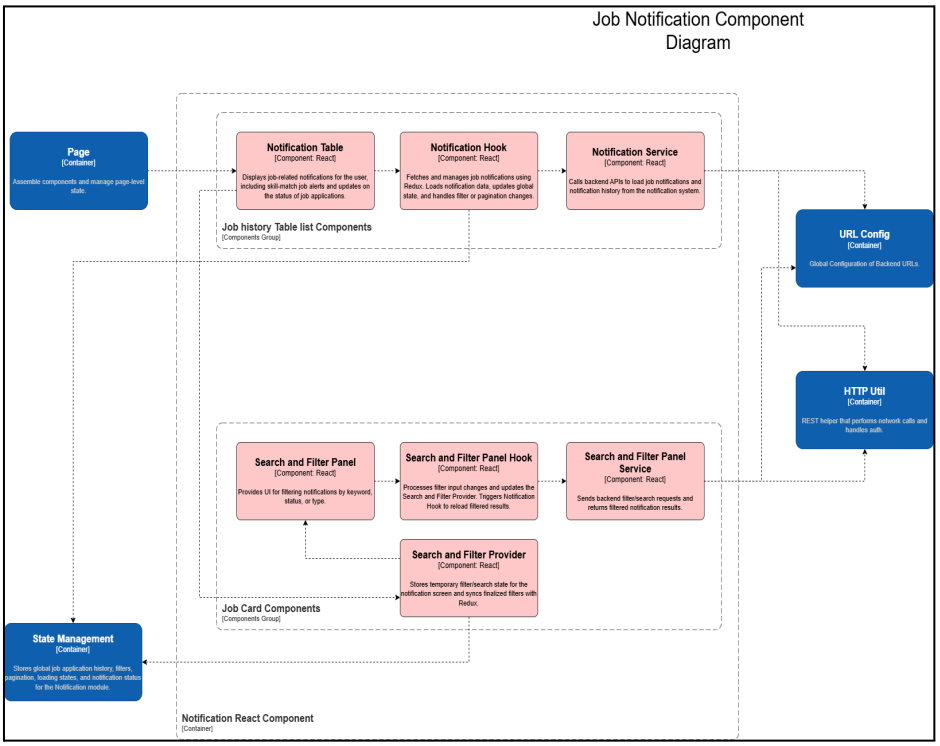

Figure 29: Frontend Notification Component Diagram

The **Notification Component** follows the same modular architecture as other JA Frontend modules, separating presentation, logic, backend communication, filtering, and global state management. This ensures the notification system remains scalable and consistent with the React–Redux design of the platform. The main objective of this module is to deliver personalized and real-time job-related notifications to users based on their account tier and activity.

At the presentation layer, the **Notification Table** displays all notifications relevant to the user. For regular accounts, the table shows **application status updates** such as “Pending,” “Under Review,” “Accepted,” or “Rejected.” These notifications help users track the progress of their job applications. For VIP accounts, the system additionally generates **skill-match job alerts** when new job postings share required skills with the user’s profile. This tier-based behavior allows the platform to provide enhanced value to premium users while keeping the core experience streamlined for regular users.

Below the UI layer, the **Notification Hook** manages the core logic of the module. It loads notifications by reading filter values from Redux, calls the Notification Service to fetch data from the backend, and updates global state. It also handles pagination, loading states, and reloading when users apply filters or change notification categories. This keeps the UI stateless and ensures predictable data flow.

The module includes a **Search and Filter Panel**, allowing users to refine notifications by keyword, job status, or notification type (status update vs. skill-match alert). The **Search and Filter Panel Hook** processes user input, updates the Search and Filter Provider, and triggers the Notification Hook to refresh results. This provides an efficient filtering experience without unnecessary global state updates.

Backend communication is handled by the **Notification Service**, which retrieves the user's notification history and supports server-side filtering when required. All API calls are performed through shared utilities, including **HTTP Util** for authentication and error handling, and **URL Config** to ensure consistent endpoint configuration across environments.

Global state is managed through **Redux**, which stores notification data, filters, pagination, and loading indicators. Temporary filter states remain in the **Search and Filter Provider** until finalized and synced with Redux. Through this layered and modular approach, the Job Notification component provides a reliable, user-specific notification experience—ranging from status updates for regular users to advanced skill-match alerts for VIP accounts.

Admin Panel React Component

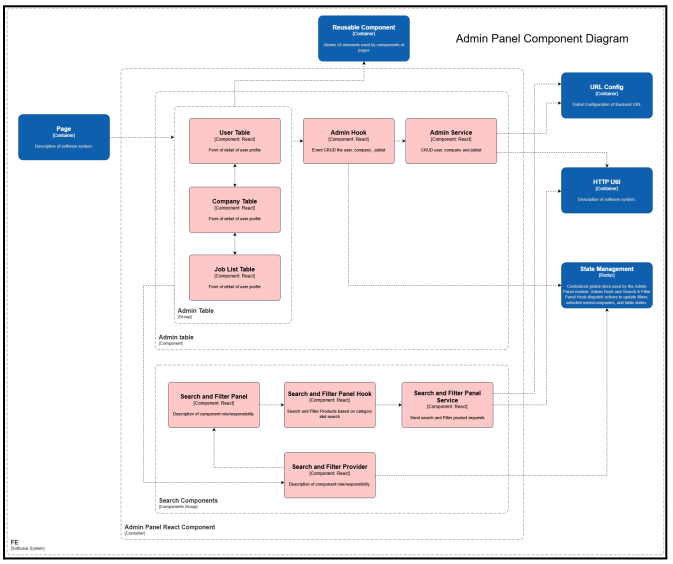

Figure 30: Frontend Admin Panel Component Diagram

At the presentation layer, the Admin Panel module includes the **User Table, Company Table, Job List Table**, and a **shared Search and Filter Panel**.

User Table displays a structured list of all user accounts. It presents key user fields and allows the administrator to select a user for review or editing.

Company Table provides an interface for browsing and managing company accounts. It displays important company attributes and offers entry points for editing, suspending, or deleting company profiles.

Job List Table allows administrators to view all job posts in the system. It presents job titles, associated companies, categories, and publication status. The focus of this component is to visualize job post data without containing business logic.

Search and Filter Panel provides a consistent interface for filtering users, companies, and job posts. Administrators can refine results based on attributes such as status, date range, job category, or company type.

Beneath the UI, the **Logic Layer** uses a set of dedicated hooks to handle interaction logic:

Admin Hook

- Handles CRUD actions initiated by table components.
- Coordinates updates, deletions, and detail viewing operations.

Search and Filter Panel Hook

- Processes filter input from the UI.
- Synchronizes selected filters with global application state.

Service Layer includes dedicated service components for handling backend communication:

Admin Service

- Executes CRUD functions (user, company, job post).
- Sends all administrative requests through the backend API.

Search and Filter Panel Service

- Performs server-side search and filtering when required.
- Retrieves filtered datasets for tables.

All service components depend on **shared foundational modules**:

HTTP Util standardizes REST communication by attaching authentication headers, handling error responses, and applying consistent request behavior across the module.

URL Config defines the backend endpoints used for administrative functions, ensuring consistent routing and environment-based configuration.

To support consistent filtering and administrative behavior, the Admin Panel module integrates with **State Management**:

Redux - Global State

- Stores admin filters, selected items, and shared UI states.
- Ensures consistent admin behavior across table components.

Search and Filter Provider

- Maintains temporary UI-level filter values.
- Works together with Redux to provide smooth filtering and reduce unnecessary re-renders.

Subscription React Component

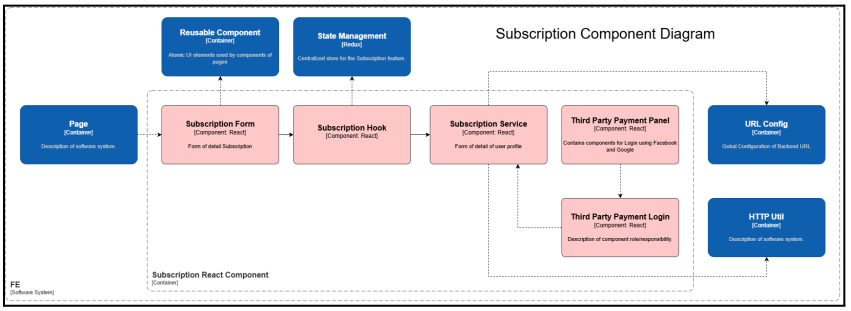

Figure 31: Frontend Subscription Component Diagram

At the presentation layer, the Subscription module consists of the Subscription Form, Third-Party Payment Panel, and Third-Party Payment Login components.

Subscription Form provides the user interface for selecting or upgrading a subscription plan. It presents available plan options and collects the user's confirmation before initiating the payment workflow.

Third-Party Payment Panel displays the available external payment providers. It offers payment options (e.g., PayPal or Stripe) and guides users into initiating secure off-site payment flows.

Third-Party Payment Login manages provider-specific interactions, such as redirecting users to the external payment gateway, confirming authorization, and returning payment results back to the Subscription module.

Beneath the UI, the module uses a set of hooks to manage user interaction and coordinate subscription logic:

The **Subscription Hook** orchestrates the complete subscription lifecycle. It receives user selections from the Subscription Form, retrieves user information from State Management, and triggers the Subscription Service to start payment sessions or fetch current subscription status. It also manages loading and error states to keep the user informed during the process.

The **Third-Party Payment Hook** manages events arising from external payment interactions. It handles redirect callbacks, processes provider responses, and ensures the Subscription Service receives the necessary payment confirmation details.

Backend communication is encapsulated within dedicated service components:

The **Subscription Service** coordinates subscription-related backend operations. It creates payment sessions, validates successful payments, updates subscription status, and retrieves the user's active plan from the backend. This service ensures a consistent flow between the frontend, API Gateway, and third-party providers.

The **Third-Party Payment Service** handles provider-specific interactions that require backend approval or validation. It communicates with the backend to confirm external payment results before updating the local subscription state.

All services rely on two **shared foundational modules**:

HTTP Util standardizes all network communication by attaching authentication information, handling REST responses, and ensuring a consistent request pattern across payment workflows.

URL Config provides centrally managed backend endpoint paths so that all subscription and payment requests route correctly across development, staging, and production environments.

To maintain consistent subscription information across the application, the module integrates with **State Management (Redux)**. Global subscription state—including plan type, activation status, expiration, and pending payment intent—is stored centrally. The Subscription Hook updates this state, while UI components read from it to present accurate subscription details. This ensures that the subscription experience remains consistent even when navigating across pages or refreshing the application.

Architecture Rationale

This section explains the rationale behind the Data Model, Frontend, and Backend architecture of the DEVision Job Applicant Platform. Each architectural property—Maintainability, Extensibility, Resilience, Scalability, Security, and Performance—is evaluated through advantages and disadvantages relevant to the system's context.

Data Model Justification

Maintainability

- *Pros:* MongoDB's flexible schema allows easy updates to user profiles, applications, and subscriptions without migrations.
- *Cons:* Without strict validation, collections may become inconsistent over time.

Extensibility

- *Pros:* New fields (e.g., skill tags, preferences) can be added without breaking existing data.
- *Cons:* Over-flexibility can lead to uncontrolled schema growth.

Resilience

- *Pros:* Replication ensures availability if nodes fail.
- *Cons:* Requires careful index tuning to avoid degraded performance under load.

Scalability

- *Pros:* Sharding supports high-volume job data and notification logs.
- *Cons:* Poorly selected shard keys may create hotspots.

Security

- *Pros:* Access control and encryption protect sensitive user data.
- *Cons:* Misconfigured permissions may expose collections.

Performance

- *Pros:* Fast reads optimized for job browsing and application queries.
- *Cons:* Large embedded documents may slow updates.

Frontend Architecture Justification

Maintainability

- *Pros:* Modular architecture (JobList, Profile, Notification) isolates code for easier updates.
- *Cons:* Too many modules require strict team conventions.

Extensibility

- *Pros:* New features like Recommendations or Interview Tracking can be added as new modules.
- *Cons:* Expanding global Redux state may require refactoring.

Resilience

- *Pros:* Redux and error boundaries keep the UI stable during backend failures.
- *Cons:* UI still depends on backend availability.

Scalability

- *Pros:* Lazy loading and reusable components support large user bases.
- *Cons:* Complex component trees require careful optimization.

Security

- *Pros:* HTTP-only cookies and role-based access restrict sensitive actions.
- *Cons:* Client-side code visibility prevents storing sensitive logic on FE.

Performance

- *Pros:* Pagination, memoization, and caching accelerate job searches.
- *Cons:* Inefficient state updates can cause unnecessary re-renders.

Backend Architecture Justification

Maintainability

- *Pros:* Microservices (Auth, Profile, Application, Matching, Subscription, Notification Consumer) follow single responsibility, simplifying testing and upgrades.
- *Cons:* More services increase operational complexity.

Extensibility

- *Pros:* New services such as Analytics or Recommendation can be added without modifying existing ones.
- *Cons:* Requires API versioning to avoid breaking clients.

Resilience

- *Pros:* Independent services prevent entire system failures; Kafka provides durable event-driven notifications.
- *Cons:* Network delays between services may propagate issues.

Scalability

- *Pros:* Each service scales independently based on traffic patterns.
- *Cons:* Infrastructure overhead grows with more services.

Security

- *Pros:* API Gateway centralizes authentication, rate-limiting, and service isolation.
- *Cons:* Misconfigured gateway rules may expose internal endpoints.

Performance

- *Pros:* Services optimized individually; Kafka enhances VIP job alert performance.
- *Cons:* Multiple synchronous API calls may increase latency

Conclusion

The DEVision Job Applicant System delivers a complete set of features supporting the applicant journey, including user authentication, profile management, job search and filtering, job application submission, application status tracking, premium subscription handling, and both regular and VIP notification services. VIP users benefit from skill-match job alerts powered by the event-driven backend, while all users receive updates on their application status. The system integrates a modular frontend, a microservices backend, an API Gateway for secure communication, and a flexible MongoDB data model, together forming a scalable and modern platform.

Despite these strengths, the system also presents several challenges. The flexibility of the document data model can introduce inconsistencies without strict validation. The microservices architecture increases deployment and monitoring complexity, and cross-service communication may lead to latency if not optimized. On the frontend, extensive use of Redux can cause unnecessary re-renders without careful state management. These limitations highlight the need for disciplined engineering practices to maintain system reliability, performance, and long-term sustainability.